

Chronos

100 Viva and Cross-Questions

Concise 1-2 line answers for the final project defense

Coverage: problem statement, fixed-depth Merkle hash trie, incremental hashing, SQLite persistence, hybrid diff, API/frontend, experimental design, Dolt comparison, results, literature review, limitations, and future work.

1. Project Fundamentals	Questions 1-10
2. Data Structure and Design Choices	Questions 11-20
3. Incremental Commit and Persistence	Questions 21-30
4. Diff, History, and Checkout	Questions 31-40
5. Backend, API, and Frontend	Questions 41-50
6. Experimental Design	Questions 51-60
7. Metrics, Statistics, and Evidence	Questions 61-70
8. Results and Interpretation	Questions 71-80
9. Sensitivity Study and Literature Review	Questions 81-90
10. Difficult Defense Questions, Limitations, and Future Work	Questions 91-100

Defense rule: report measured results as system-level observations, not universal algorithmic superiority. The final evidence reaches 100,000 rows, and the current bibliography contains 11 scholarly works plus four primary technical sources.

1. Project Fundamentals

Q1. What is Chronos in one sentence?

A. Chronos is a content-addressed versioned store for structured key/value data, using immutable commits and an incrementally updated Merkle hash trie.

Q2. What problem does Chronos solve?

A. It preserves exact dataset versions while supporting sparse updates, historical checkout, integrity verification, and efficient comparison without copying every full state.

Q3. Why is Chronos described as Git-inspired?

A. It borrows immutable objects, content-derived identities, parent-linked commits, HEAD, history, checkout, and diff; unlike Git, it versions structured records rather than files.

Q4. Why is ordinary file version control insufficient for structured data?

A. File-oriented tools do not naturally expose record-level updates, typed change categories, key-based queries, or database-style version comparisons.

Q5. What does content addressing mean?

A. An object's identifier is computed from its canonical content. Equal objects have equal hashes, enabling deduplication, structural sharing, and corruption checks.

Q6. What role does a Merkle structure play?

A. Parent hashes summarize their descendants, so equal hashes prune unchanged regions and the root commits to the complete logical state.

Q7. What is a version in Chronos?

A. A version is a commit object containing a root hash, parent commit, changeset, message, and timestamp; user-friendly aliases such as v1 point to commits.

Q8. What is the main research contribution?

A. The contribution is an instrumentable hybrid design combining incremental fixed-trie hashing with adaptive operation-log or Merkle differencing.

Q9. What is Chronos-H?

A. Chronos-H is the primary hybrid variant. It selects log aggregation for short or sparse intervals and Merkle comparison once accumulated operations exceed a calibrated boundary.

Q10. Why was Python retained?

A. Python enabled a complete, testable prototype within the deadline and made instrumentation simple; runtime asymmetry with Dolt is explicitly treated as a limitation.

2. Data Structure and Design Choices

Q11. What data structure is actually implemented?

- A. The evaluated core is a fixed-depth Merkle hash trie: SHA-256 bits route keys through a fixed number of levels, and leaves retain complete keys and values.

Q12. Why did you not switch to a Go Prolly Tree?

- A. A late rewrite would risk correctness and reproducibility. The fixed trie provides a controlled research design, while Dolt already supplies the production Prolly Tree comparison.

Q13. Is Chronos identical to the published Merkle Search Tree?

- A. No. It is Merkle-search-tree-inspired but uses fixed-depth hash routing, not the ordered balanced MST of Auvolat and Taiani; the distinction must be stated clearly.

Q14. What are the default trie parameters?

- A. The evaluated default uses branching factor $b=8$ and depth $d=4$, consuming three routing bits per level.

Q15. Why does $b=8$, $d=4$ produce 4,096 buckets?

- A. The number of possible leaf routes is b^d , so 8^4 equals 4,096; equivalently, twelve hash-routing bits are consumed.

Q16. What happens when two keys share the same routing prefix?

- A. Both remain as distinct full-key entries in the same leaf bucket. A routing collision affects occupancy and work, not correctness.

Q17. What does the root hash represent?

- A. It is a compact commitment to the complete trie state. Any changed record propagates new hashes from its leaf to the root.

Q18. Does the same logical state always have the same root?

- A. Yes, provided serialization and trie parameters are identical. Canonical encoding and deterministic routing make the state root reproducible.

Q19. Does the same state always produce the same commit hash?

- A. Not necessarily. A commit also includes its parent, changeset, message, and timestamp, so identical roots can belong to different historical commits.

Q20. What is structural sharing?

- A. A new version reuses unchanged content-addressed nodes from its parent and creates only affected leaves and ancestors, reducing duplicate storage.

3. Incremental Commit and Persistence

Q21. What is incremental hashing?

A. Chronos recomputes only changed leaf buckets and their ancestor paths rather than rebuilding and rehashing the entire tree for every commit.

Q22. Which nodes change after one key update?

A. The affected leaf and one ancestor at each level up to the root change; unrelated subtrees keep their existing hashes and stored objects.

Q23. Why group mutations by shared prefixes?

A. It prevents rewriting the same leaf or ancestor repeatedly when several keys in one batch follow overlapping trie paths.

Q24. Why is canonical JSON necessary?

A. Hashing requires one stable byte representation. Sorted fields and consistent encoding prevent semantically identical objects from receiving different hashes.

Q25. Why use SHA-256?

A. It is widely available, deterministic, and collision resistant enough for prototype integrity and content identity; the design does not depend on secret-key security.

Q26. What is stored by hash?

A. Trie nodes, changesets, and commit objects are immutable addressed objects; aliases, HEAD, and operational measurements are stored as repository metadata.

Q27. Why use SQLite?

A. SQLite offers zero-configuration durable persistence and transactions, matching a local prototype while keeping experiments reproducible and easy to reopen.

Q28. How is a commit made atomic?

A. New objects, changeset, commit metadata, version alias, measurements, and the HEAD update are written inside one SQLite transaction.

Q29. What is HEAD?

A. HEAD is the durable named reference to the latest commit. It is metadata, not the identity of the underlying state.

Q30. How does Chronos detect stored corruption?

A. When reopening, it recomputes object hashes from stored payloads and checks references; a mismatch causes the repository to be rejected.

4. Diff, History, and Checkout

Q31. How does Merkle diff work?

- A. It compares roots, prunes equal subtree hashes, recursively follows unequal paths, and inspects full leaf entries only below changed regions.

Q32. How does log diff work?

- A. It walks the ancestor chain and aggregates addressed changesets, preserving the net added, deleted, and updated records between two versions.

Q33. Why combine log and Merkle diff?

- A. Logs are efficient for short sparse histories, while Merkle pruning avoids replaying long histories when many operations accumulated between endpoints.

Q34. What is the current hybrid threshold?

- A. The frozen experimental boundary is 4,096 accumulated operations between versions; it is a measured policy for one environment, not a universal constant.

Q35. Is 4,096 included in the log path?

- A. Yes. Chronos-H chooses log aggregation at or below 4,096 operations and forced-Merkle comparison above it.

Q36. Can Chronos compare versions in reverse order?

- A. Yes. The reverse operation swaps additions and deletions and reverses old/new values for updates while preserving logical correctness.

Q37. What is Chronos-M?

- A. Chronos-M is a forced-Merkle ablation that uses the same durable trie as Chronos-H but disables adaptive selection, isolating the selector's effect.

Q38. What happens on a no-op commit?

- A. The root is reused, an empty addressed changeset is recorded, and a new historical commit may still be created without duplicating trie state.

Q39. How does checkout work?

- A. Chronos resolves the version to its root and materializes all key/value records reachable from that immutable trie state.

Q40. What is the expected diff complexity?

- A. It depends on changed paths and bucket occupancy, not only total records; the implementation reports examined node pairs rather than claiming one universal bound.

5. Backend, API, and Frontend

Q41. What are the main system layers?

- A. The platform consists of a metrics-focused frontend, REST repository service, Chronos-H core, and SQLite content-addressed object store.

Q42. Which backend operations are exposed?

- A. The API supports create/open repository, import dataset, commit batch, history, checkout, version comparison, storage metrics, and diff metrics.

Q43. What is the frontend's purpose?

- A. It demonstrates the versioning workflow and presents evidence such as latency, storage, history, strategy choice, and comparison results in one workspace.

Q44. Why use a metrics-first interface?

- A. The project's main claim is experimental, so the interface prioritizes measurable behavior and reproducible comparisons over decorative application features.

Q45. What do the 10, 100, 10k, 100k, and 1m controls mean?

- A. They are workload-size selectors in the interface; only sizes present in the audited benchmark may be presented as measured research evidence.

Q46. How does the API protect correctness?

- A. It validates payloads and base versions, rejects duplicate or invalid operations, and returns structured errors without partially committing data.

Q47. Why support CORS and preflight requests?

- A. They let the separately hosted frontend call the local API safely through standard browser cross-origin controls.

Q48. How are unsafe repository names handled?

- A. Names are validated so they cannot escape the configured repository root through path traversal or malformed filesystem paths.

Q49. Why not PostgreSQL for this prototype?

- A. PostgreSQL would help multi-user concurrency and deployment, but SQLite is sufficient for a local atomic store and reduces operational variables in experiments.

Q50. What would the backend need for production use?

- A. It would need authentication, authorization, concurrency control, schema evolution, branch/merge semantics, remote synchronization, backups, and stronger observability.

6. Experimental Design

Q51. Which five variants are compared?

A. Snapshot, Log-only, forced-Merkle Chronos-M, adaptive Chronos-H, and Dolt 2.3.1 receive equivalent logical workloads.

Q52. Why include a Snapshot baseline?

A. It represents complete state materialization per version, providing a simple reference for storage, import, checkout, and full-scan diff behavior.

Q53. Why include a Log-only baseline?

A. It isolates the benefits and costs of storing an initial state plus operations, especially for sparse histories and reconstruction-heavy workloads.

Q54. Why compare with Dolt?

A. Dolt is a mature version-controlled SQL database using content-addressed Proly Trees, making it the most relevant production system-level comparator.

Q55. How is fairness maintained across implementations?

A. All adapters receive regenerated workloads with identical logical states, mutation batches, seeds, and correctness oracles, while interface differences are disclosed.

Q56. What workloads are evaluated?

A. The paper uses small sparse, medium sparse, medium dense, large sparse, and large hot scenarios up to 100,000 rows.

Q57. How many measured repetitions are used?

A. Each final scenario uses seven measured trials after two warmups, and reported medians and quartiles exclude warmups.

Q58. Why use warmups?

A. Warmups reduce one-time startup, import, cache, and process initialization effects from contaminating the measured trial summaries.

Q59. Why use fixed random seeds?

A. A fixed base seed makes workloads reproducible; SHA-256 workload digests confirm that every adapter receives the intended mutations.

Q60. How is correctness established?

A. Every final state and diff is checked against an independent changed-key oracle, with successful execution alone never treated as proof of correctness.

7. Metrics, Statistics, and Evidence

Q61. Which metrics are recorded?

A. Initial import, incremental commit, diff, checkout, repository storage, peak memory where comparable, work examined, selected strategy, and correctness.

Q62. Why report medians and interquartile ranges?

A. Short system timings can be noisy; the median is robust to extreme values, while the 25th-75th percentile range shows dispersion.

Q63. What does nodes or operations examined mean?

A. It reports each implementation's natural internal work unit; it is diagnostic and is not normalized as a cross-system equivalent counter.

Q64. How is storage measured?

A. Storage is the sum of regular repository-file bytes after the evaluated history, so indexes, metadata, and content-addressed objects contribute.

Q65. Why is there no cross-system memory graph?

A. Python tracemalloc and Dolt process memory are not equivalent measurement methods; presenting them together would imply false comparability.

Q66. Were outliers removed?

A. No. The audit identified 83 Tukey candidates, retained every observation, and summarized all measured trials with medians and quartiles.

Q67. What does the 333-trial figure mean?

A. The audited final paper profile contains 333 successful, correctness-true measurement rows across calibration and evaluation combinations.

Q68. What does the 45-trial sensitivity figure mean?

A. Five trie configurations were evaluated with nine runs each, producing 45 correctness-verified sensitivity observations.

Q69. Why separate calibration from evaluation?

A. A disjoint calibration workload freezes the hybrid policy before evaluation and reduces the risk of tuning directly to headline results.

Q70. Do seven trials prove statistical significance?

A. No. They support descriptive medians and quartiles, but not strong distributional or population claims; more machines and repetitions are future work.

8. Results and Interpretation

Q71. What is the main diff result against Dolt?

A. Chronos-H's measured CLI-facing median diff latency was 5.8x to 554.2x lower across the five final workloads.

Q72. What is the incremental commit result?

A. Chronos-H's incremental commit medians were 2.3x to 37.4x lower than Dolt under the evaluated protocol.

Q73. What is the checkout result?

A. Chronos-H's historical checkout medians were 35.8x to 123.4x lower than Dolt in the tested workloads.

Q74. Did Chronos-H always import faster?

A. No. It was faster on small sparse, roughly tied around medium dense, and about 2.73x to 2.79x slower at 100,000 rows.

Q75. What storage cost did Chronos-H pay?

A. Its final repository footprint ranged from 1.83x to 6.43x Dolt's, reflecting JSON objects, fixed buckets, and no packing, compression, or garbage collection.

Q76. Why was the largest diff ratio so high?

A. Sparse histories let the log path touch very little data while the measured Dolt path includes CLI and SQL invocation overhead; it is not a pure algorithm-only speedup.

Q77. What happened in the medium-dense workload?

A. Both Chronos-H and Chronos-M selected or used Merkle comparison, and Chronos-H stayed within about 7.3% of the forced-Merkle variant.

Q78. Which strategy did Chronos-H select?

A. It selected log for the four sparse or hot workloads and Merkle for the 10,000-operation medium-dense interval.

Q79. Does the result prove 4,096 is optimal?

A. No. It is the largest sampled log-favorable calibration point on one machine, not a precisely measured universal crossover.

Q80. What result has the strongest publication value?

A. The most defensible result is the measured trade-off: hybrid selection greatly reduces sparse diff work while fixed-trie storage and import costs remain visible.

9. Sensitivity Study and Literature Review

Q81. Why run a trie-parameter sensitivity study?

A. It prevents presenting $b=8$, $d=4$ as universally optimal and shows how depth and branching alter latency, bucket occupancy, node work, and storage.

Q82. Which parameter combinations were tested?

A. The study evaluates five controlled branching-factor/depth configurations around the default, using identical workloads and seeds.

Q83. Why retain $b=8$, $d=4$ as the default?

A. It used the least storage in the tested set, about 4.27 MiB, while providing a balanced rather than latency-only operating point.

Q84. What advantage did $b=8$, $d=3$ show?

A. It had the lowest median diff latency, 20.15 ms versus 44.43 ms for $b=8$, $d=4$, and compared 84.3% fewer node pairs.

Q85. What was the cost of $b=8$, $d=3$?

A. It consumed 61.2% more storage and examined about three times as many leaf entries because its shallower buckets were larger.

Q86. What did the over-deep configuration show?

A. The $b=8$, $d=5$ design was slower for every timed operation and used 44.8% more storage, showing that additional depth is not automatically beneficial.

Q87. What is the sensitivity-study conclusion?

A. Trie geometry must be chosen by workload objective: shallow routing may favor latency, while deeper routing can reduce occupancy but increase internal-node work and storage.

Q88. What does the literature review cover?

A. It reviews content addressing and persistent structures, structured dataset versioning, and the design gap between research prototypes and Dolt's production Prolly Trees.

Q89. Are all 15 bibliography entries research papers?

A. No. Eleven are scholarly papers or preprints; Noms documentation, Pro Git, Dolt documentation, and a DoltHub article are primary technical sources.

Q90. What bibliography corrections were found?

A. The MST DOI, Silu Huang's name in two entries, Liqi Xu's name, and missing DOIs for dataset versioning, OrpheusDB, and LBFS require correction.

10. Difficult Defense Questions, Limitations, and Future Work

Q91. Can you claim Chronos is better than Dolt?

A. Only within the declared CLI-facing benchmark for selected operations. Dolt remains far more feature-complete, production-engineered, and storage efficient.

Q92. Is Python Chronos versus Go Dolt an apples-to-apples comparison?

A. No. It is a system-level comparison through each tool's public interface; runtime and interface asymmetry are explicit threats to validity.

Q93. What are the largest technical limitations?

A. Chronos lacks branches, merges, schema evolution, range indexes, concurrent writers, remotes, packing, compression, and garbage collection.

Q94. Why omit branches and merges from the final prototype?

A. The project prioritizes the storage model, durable version history, diff strategy, and reproducible evidence; adding incomplete semantics would dilute the core contribution.

Q95. Was one million rows benchmarked?

A. No. The audited paper evidence reaches 100,000 rows. A 1m interface option must not be described as a measured result until that experiment is completed.

Q96. What is the biggest threat to external validity?

A. Results come from deterministic synthetic workloads on one Windows machine, so real schemas, skew, platforms, caching, and longer histories may change the outcome.

Q97. What storage improvement should come first?

A. Compact binary node encoding and pack files should come first, followed by compression, content-defined value chunking, reachability analysis, and garbage collection.

Q98. What would you do with one additional week?

A. Correct the bibliography, rerun on another machine and real dataset, add one-million-row evidence, profile I/O, and prototype packed binary objects without changing semantics.

Q99. What novelty claim is safe to defend?

A. Chronos demonstrates a reproducible hybrid log/Merkle policy over an incremental fixed-trie store and quantifies its latency-storage-structure trade-offs against controlled baselines and Dolt.

Q100. What is the final takeaway?

A. Incremental hashing avoids full-tree rebuilds, and adaptive differencing can exploit sparse histories; the gain is real in the tested protocol but purchased with storage and scope trade-offs.